HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

LÝ THUYẾT MẬT MÃ

TS. TẠ KIM HUỆ
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ONE LOVE. ONE FUTURE.

Mã xác thực (Authenticated Encryption)

Recap: the story so far

Confidentiality (tính bí mật): bảo mật ngữ nghĩa chống đươcj tấn công CPA

- Bảo mật hệ mã chỉ chống được nghe lén (**eavesdropping)**

Integrity (tính toàn vẹn):

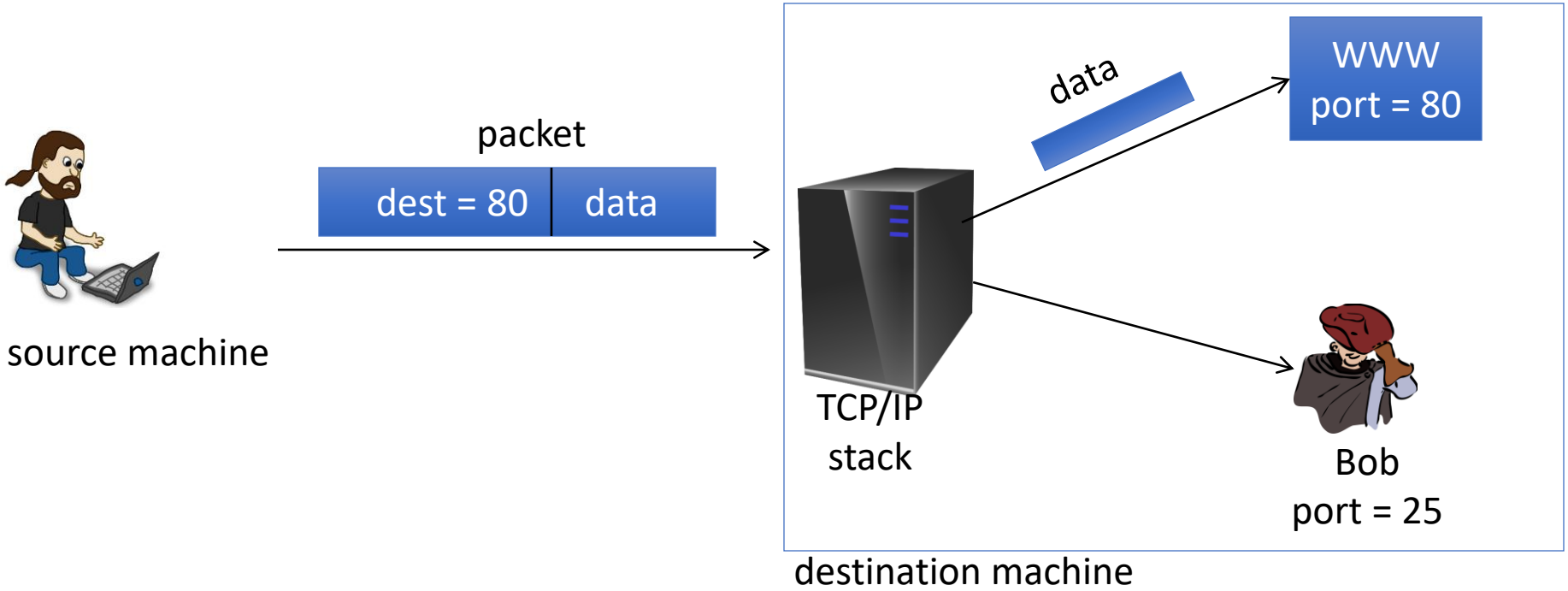
- Khả năng không thể giả mạo chống lại các tấn công lựa chọn bản rõ (Existential unforgeability under a chosen message attack)
- CBC-MAC, HMAC, PMAC, CW-MAC

This module: encryption secure against **tampering (giả mạo) (active adversary)**

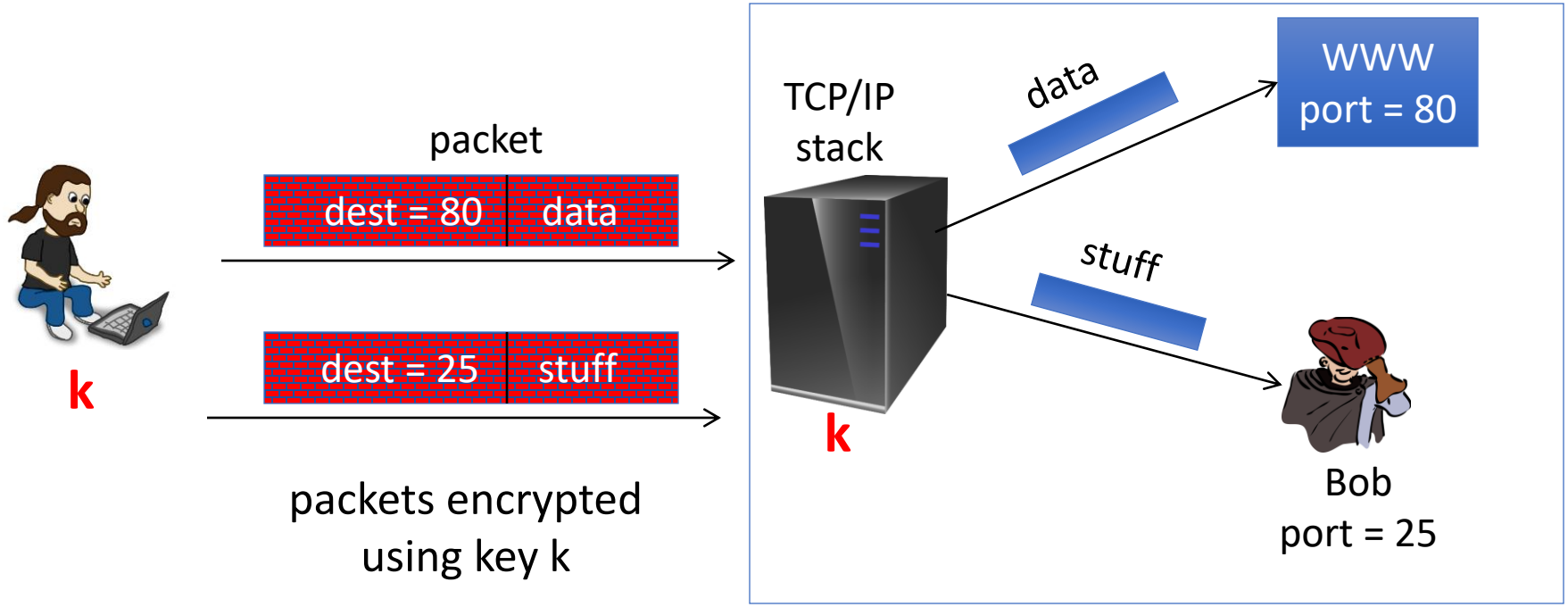
- Đảm bảo đồng thời tính bí mật và toàn vẹn (Ensuring both confidentiality and integrity)

Một mẫu tấn công giả mạo (Sample

TCP/IP: (highly abstracted -> tính trừu tượng cao)

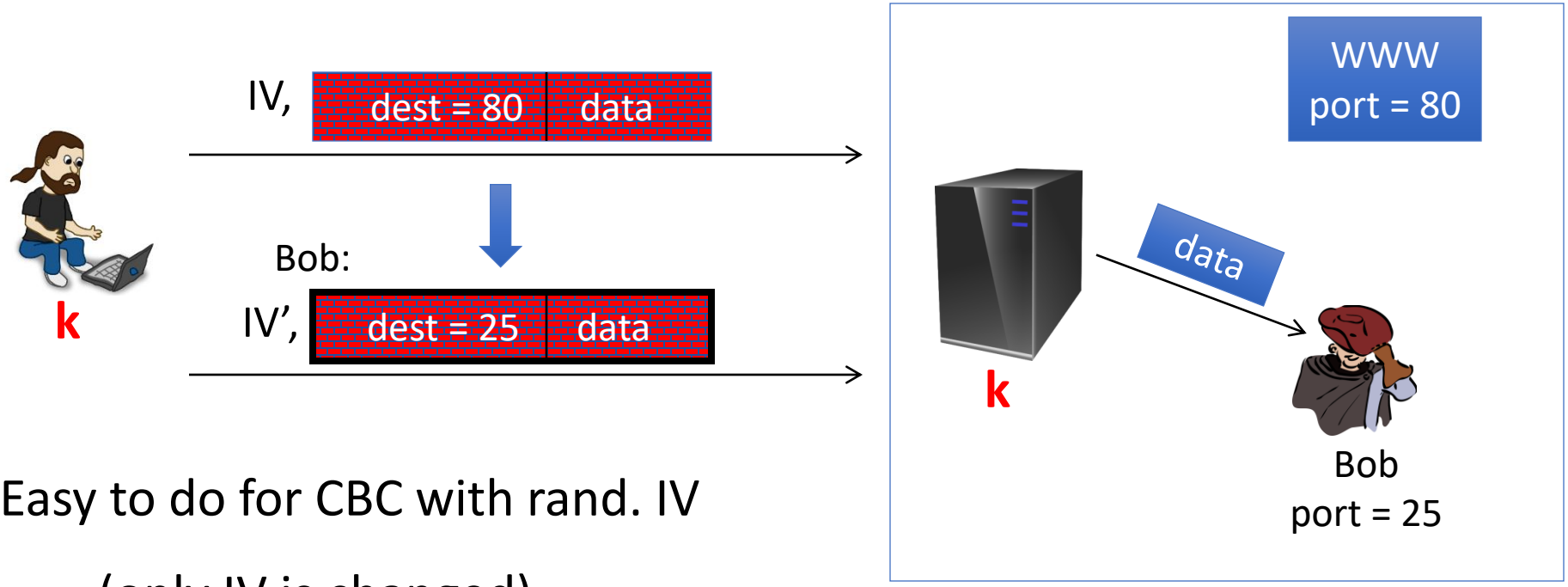


IPsec: (highly abstracted)



Reading someone else's data

Note: kẻ tấn công thu được bản giải mã của bất kỳ ciphertext bắt đầu với "dest=25"



Easy to do for CBC with rand. IV

(only IV is changed)

IV , | dest = 80 | data | → IV' , | dest = 25 | data |

Encryption is done with CBC with a random IV.

What should IV' be?

$m[0] = D(k, c[0]) \oplus IV$ = "dest=80…"

$IV' = IV \oplus (\ldots 25 \ldots)$

$IV' = IV \oplus (\ldots 80 \ldots)$

$IV' = IV \oplus (\ldots 80 \ldots) \oplus (\ldots 25 \ldots)$ ⟸

It can't be done

$\underbrace{D(k, c[0]) \oplus IV'}_{} = \overbrace{D(k, c[0]) \oplus IV \oplus \cancel{80}}^{\ldots \cancel{80} \ldots} \oplus 25$

$= \ldots 25 \ldots$

Message Auth. Codes (MAC) – Mã xác thực thông điệp

Online Cryptography Course Dan Boneh

Toàn vẹn thông điệp (Message integrity)

Toàn vẹn thông điệp (Message Integrity)

Mục tiêu: toàn vẹn (**integrity**), không bí mật bản tin (no confidentiality).

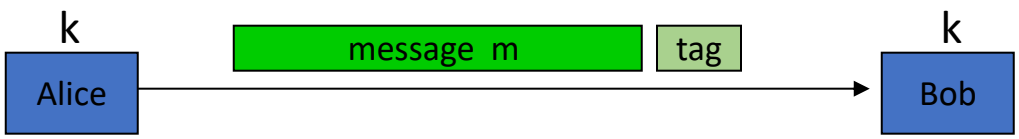
Examples:

- Bảo vệ các mã nhị phân (Protecting public binaries on disk).
- Bảo vệ các bảng quảng cáo (Protecting banner ads on web pages).

Message integrity: MACs

Def: **MAC** I = (S,V) defined over (K,M,T) is a pair of algs:

- S(k,m) outputs t in T
- V(k,m,t) outputs `yes' or `no'

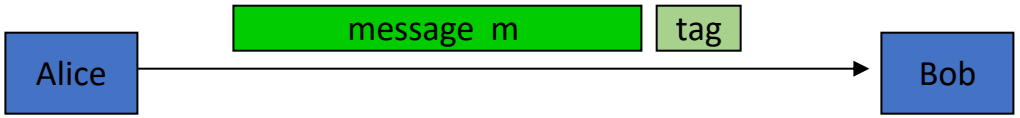


Generate tag:
tag ← S(k, m)

Verify tag:
V(k, m, tag) $\stackrel{?}{=}$ `yes'

Integrity requires a secret key

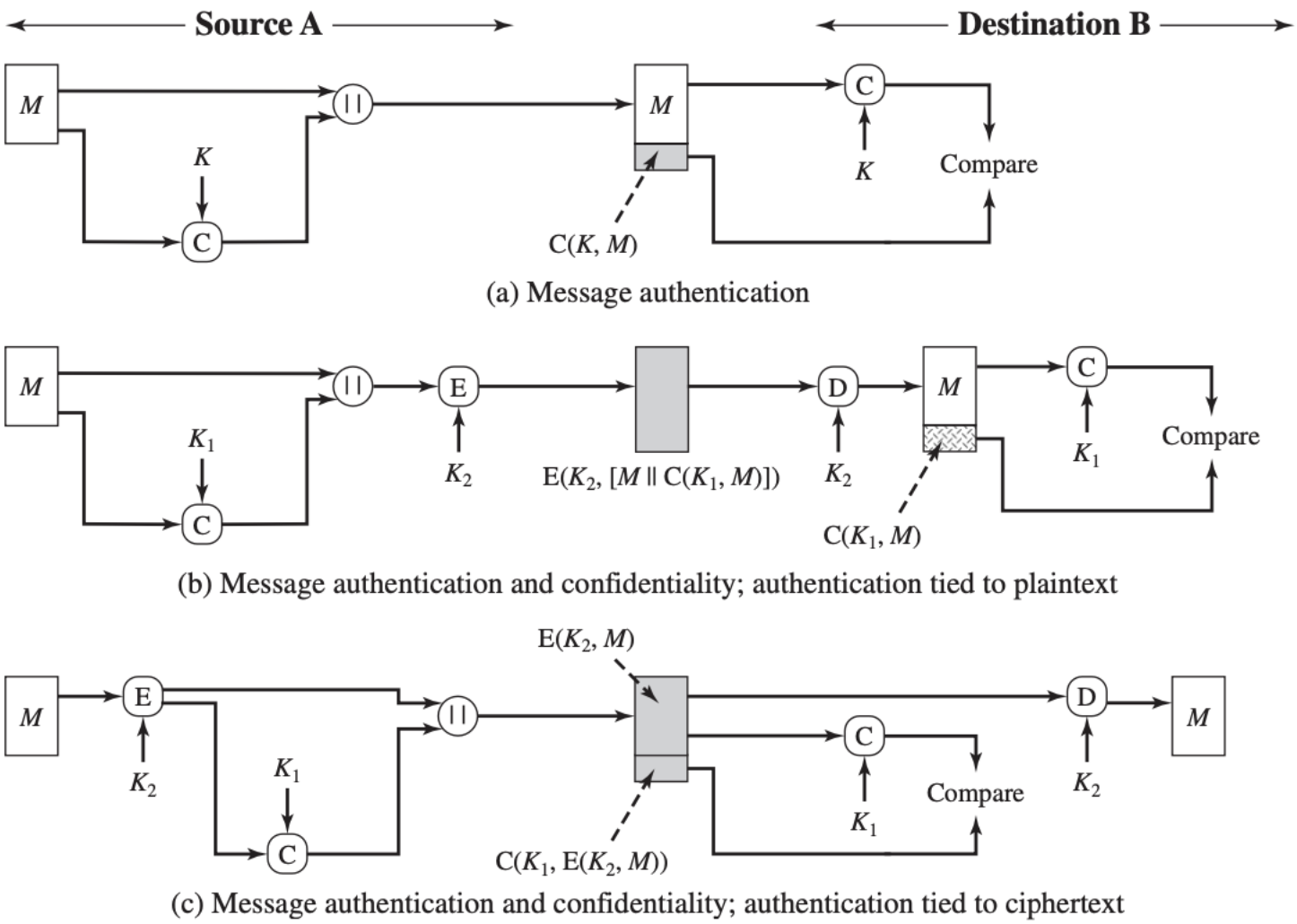
- Kẻ tấn công có thể dễ dàng sửa đổi bản tin m và tính lại mã CRC
- CRC thiết kế để kiểm tra ngẫu nhiên và không tìm được lỗi nghiêm trọng (designed to detect **random**, not malicious errors).



Generate tag:
tag ← CRC(m)

Verify tag:
V(m, tag) $\stackrel{?}{=}$ `yes'

Basic Uses of Message Authentication code (MAC)



Bảo mật MAC

Attacker's power: Tấn công chọn sẵn (**chosen message attack)**

- đưa ra các truy vấn $m_1, m_2, \ldots, m_q$ và nhận được $t_i \leftarrow S(k, m_i)$

Attacker's goal: có khả năng giả mạo (**existential forgery)**

- tạo ra cặp message/tag (m,t) hợp lệ.

$$(m,t) \notin \{ (m_1,t_1), \ldots, (m_q,t_q) \}$$

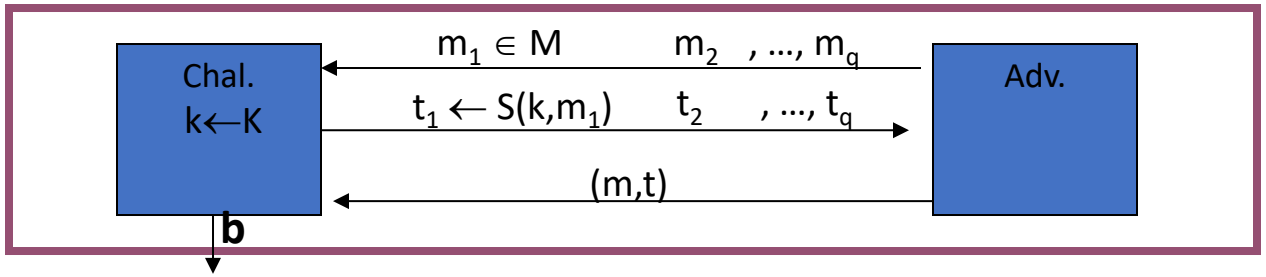
Mục tiêu ngăn chặn:

⇒ kẻ tấn công không thể tạo ra giá trị Tag hợp lệ cho thông điệp mới

⇒ nhận được (m,t) attacker không thể tạo ra (m,t') for $t' \neq t$

Bảo mật MACs

- For a MAC I=(S,V) and adv. A define a MAC game as:



$$\begin{cases} \mathbf{b}=1 & \text{if } V(k,m,t) = \text{`yes'} \text{ and } (m,t) \notin \{(m_1,t_1), \dots, (m_q,t_q)\} \\ \mathbf{b}=0 & \text{otherwise} \end{cases}$$

Def: I=(S,V) is a **secure MAC** if for all "efficient" A:

$$\mathrm{Adv}_{\mathrm{MAC}}[A,I] = \Pr[\text{Chal. outputs } 1]$$ is "negligible."

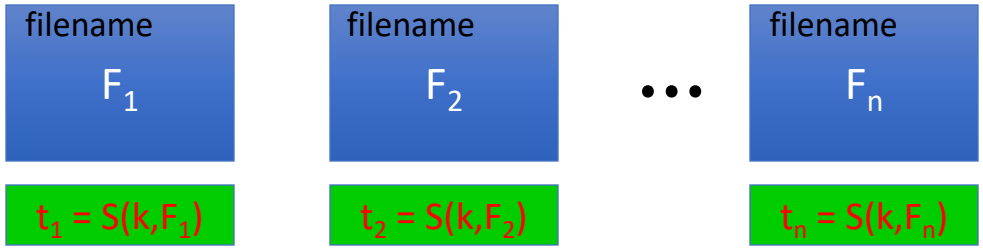
Ví dụ: protecting system files

Later a virus infects system and modifies system files

User reboots into clean OS and supplies his password

- Then: secure MAC $\Rightarrow$ all modified files will be detected

Suppose at install time the system computes:

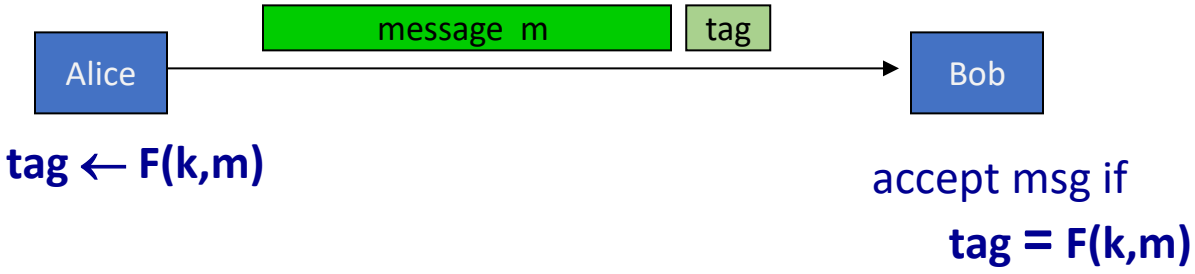


k derived from user's password

Secure PRF ⇒ Secure MAC

For a PRF **F: K × X ⟶ Y** define a MAC $I_F = (S,V)$ as:

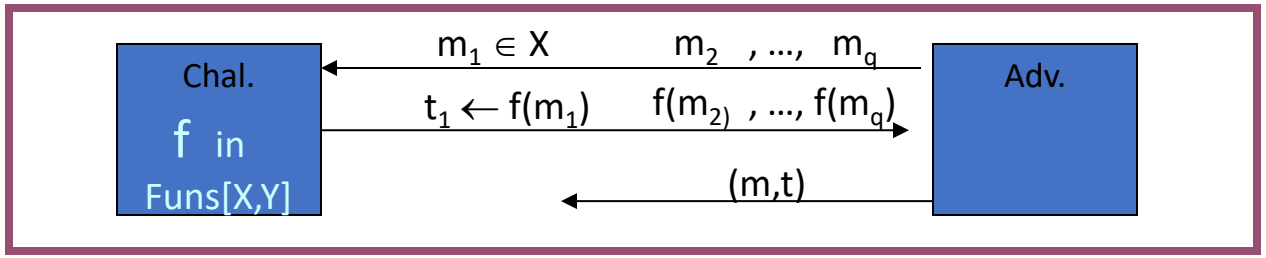
- S(k,m) := F(k,m)
- V(k,m,t): output `yes' if t = F(k,m) and `no' otherwise.



Proof Sketch

Suppose $f: X \longrightarrow Y$ is a truly random function

Then MAC adversary A must win the following game:



same must hold for F(k,x)

A wins if $t = f(m)$ and $m \notin \{ m_1 , \dots , m_q \}$

$\Rightarrow$ Pr[A wins] = 1/|Y|

Examples

- AES: có thể tạo MAC cho thông điệp 16-byte.
- Main question: bằng cách nào để chuyển đổi Small-MAC thành Big-MAC?
- Two main constructions used in practice:
  - **CBC-MAC** (banking – ANSI X9.9, X9.19, FIPS 186-3)
  - **HMAC** (Internet protocols: SSL, IPsec, SSH, …)
- Both convert a small-PRF into a big-PRF.

Recall: secure PRF **F** $\Rightarrow$ secure MAC, as long as |Y| is large

$$S(k, m) = F(k, m)$$

Our goal:

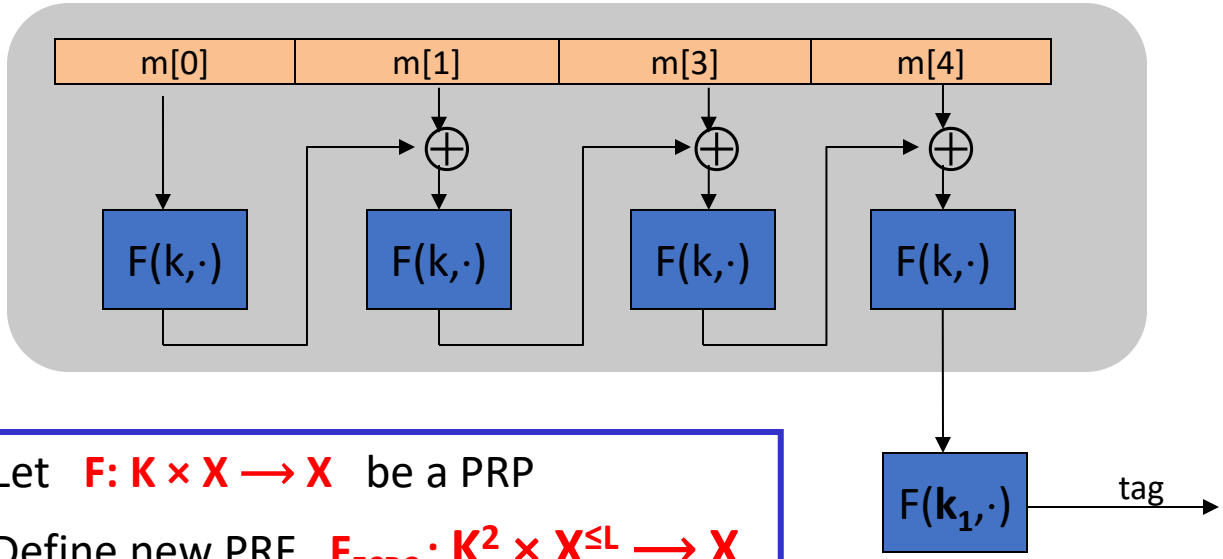
given a PRF for short messages (AES)

construct a PRF for long messages

From here on let $X = \{0,1\}^n$ (e.g. n=128)

Cấu trúc 1: encrypted CBC-MAC

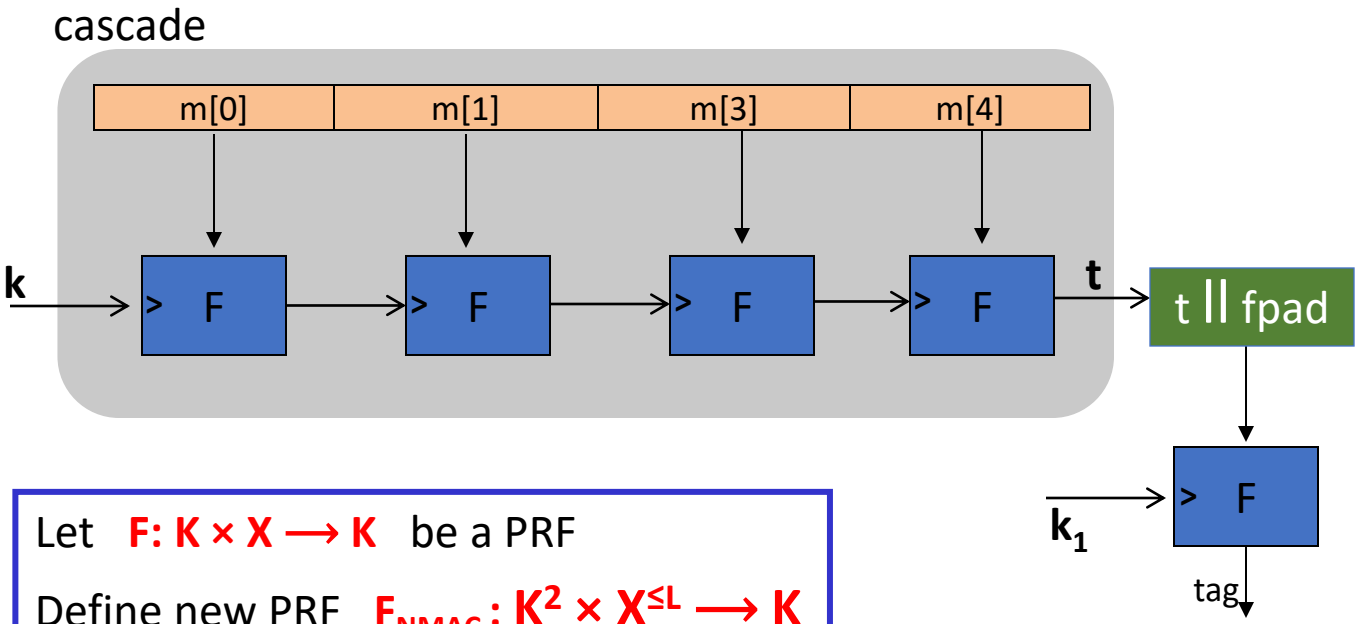
raw CBC



Let $F: K \times X \longrightarrow X$ be a PRP

Define new PRF $F_{ECBC}: K^2 \times X^{\leq L} \longrightarrow X$

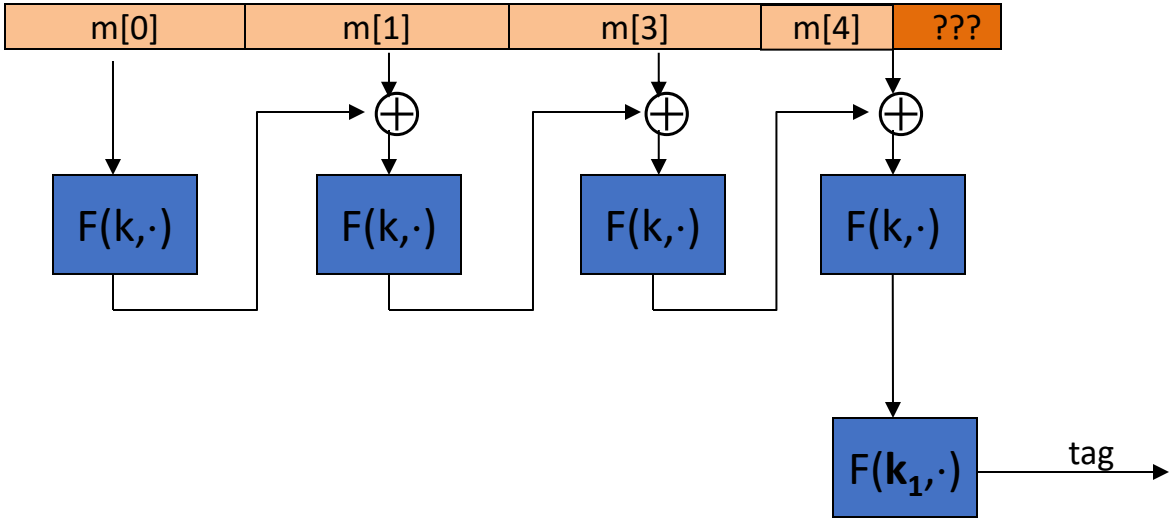
Cấu trúc 2: NMAC (nested MAC)



Let $F: K \times X \longrightarrow K$ be a PRF

Define new PRF $F_{NMAC}: K^2 \times X^{\leq L} \longrightarrow K$

What if msg. len. is not multiple of block-size?



CBC MAC padding

Bad idea: pad m with 0's

Is the resulting MAC secure?

Yes, the MAC is secure

It depends on the underlying MAC

No, given tag on msg **m** attacker obtains tag on **mll0**

Problem: pad(m) = pad(mll0)

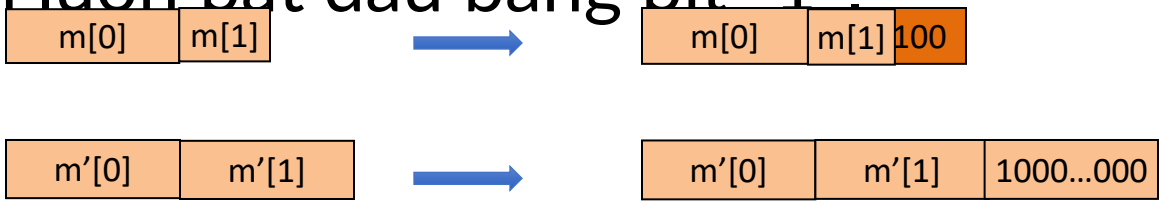
CBC MAC padding

Để bảo mật, padding (phần đệm) không thể đảo ngược!

$$m_0 \neq m_1 \quad \Rightarrow \quad pad(m_0) \neq pad(m_1)$$

ISO (tiêu chuẩn): đệm them "1000…00". Thêm một khối đệm mới nếu cần thiết

Phần đệm luôn bắt đầu bằng bit "1".

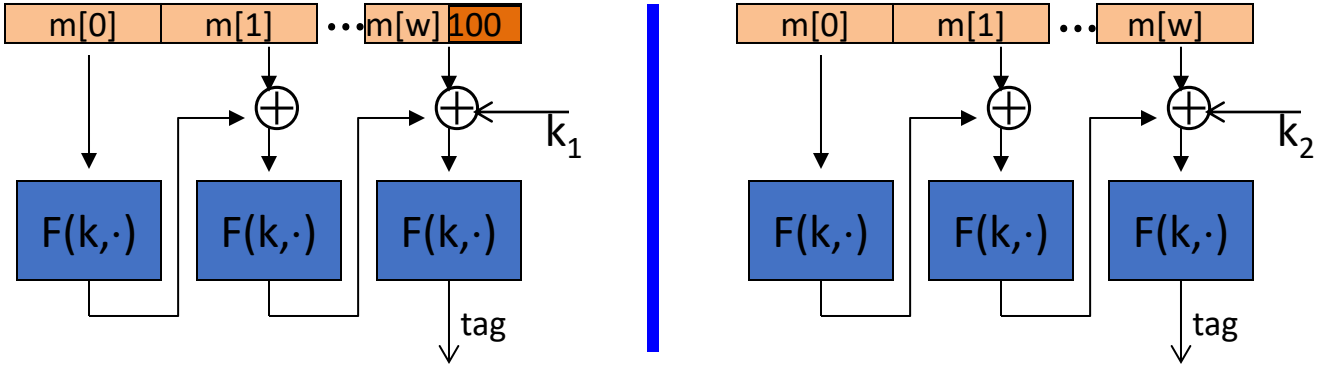


CMAC (NIST standard)

Các biến thể của CBC-MAC khi key = (k, k_1, k_2), k_1, k_2 được tạo ra từ K

No final encryption step (extension attack thwarted by last keyed xor)

- No dummy block (ambiguity resolved by use of k_1 or k_2)



MACs based on Hash function - HMAC (RFC 2104)

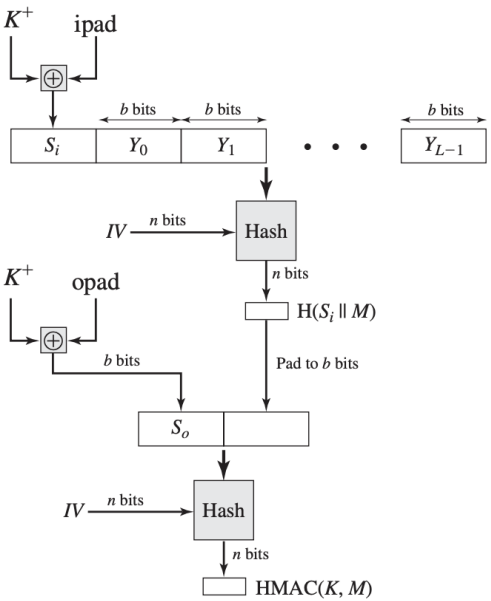
- H = embedded hash function (e.g., MD5, SHA-1, RIPEMD-160)
- *IV* = initial value input to hash function
- *M* = message input to HMAC (including the padding specified in the embedded hash function)
- Y_i = i^{th} block of *M*, 0 <= *i* <= (*L* – 1)
- *L* = number of blocks in *M*
- *b* = number of bits in a block
- *n* = length of hash code produced by embedded hash function
- *K* = secret key; recommended length is >= *n*; if key length is greater than *b*, the key is input to the hash function to produce an *n*-bit key

HMAC (RFC 2104)

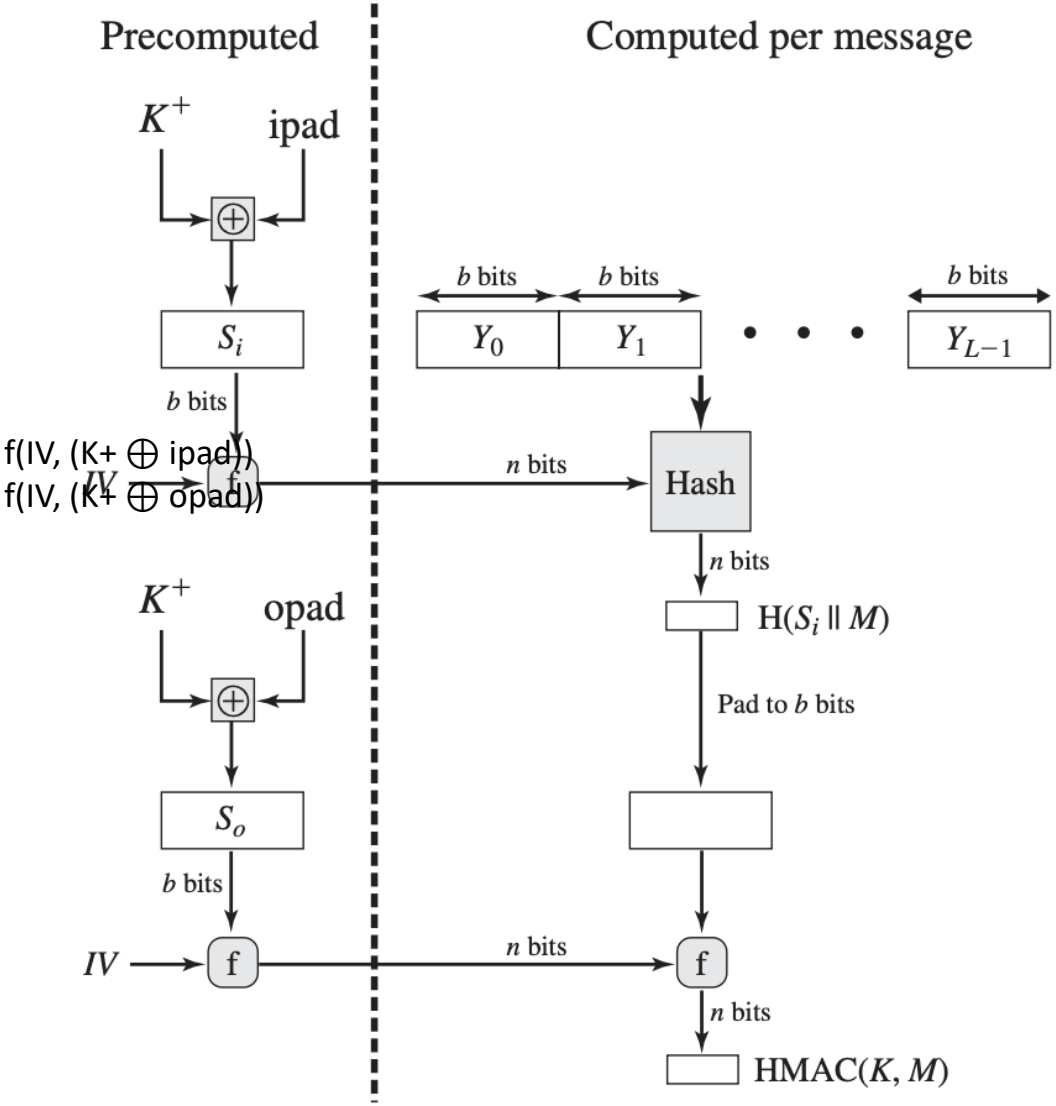
- $K+$ = K padded with zeros on the left so that the result is b bits in length
- ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times
- opad = 01011100 (5C in hexadecimal) repeated $b/8$ times
- Then HMAC can be expressed as
  $\text{HMAC}(K, M) = \text{H}[(K+ \oplus \text{opad}) \,||\, \text{H}[(K+ \oplus \text{ipad}) \,||\, M]]$

HMAC (RFC 2104)

1. Append zeros to the left end of *K* to create a *b*-bit string *K*+ (e.g., if *K* is of length 160 bits and *b* = 512, then *K* will be appended with 44 zeroes).
2. XOR (bitwise exclusive-OR) K+ with ipad to produce the b-bit block S_i.
3. Append M to S_i.
4. Apply H to the stream generated in step 3.
5. XOR K+ with opad to produce the b-bit block So.
6. Append the hash result from step 4 to So.
7. Apply H to the stream generated in step 6 and output the result.

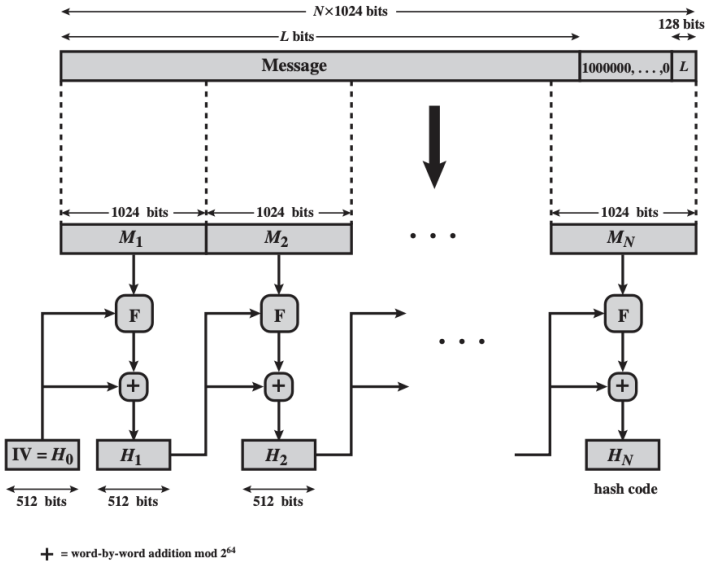


Security of HMAC

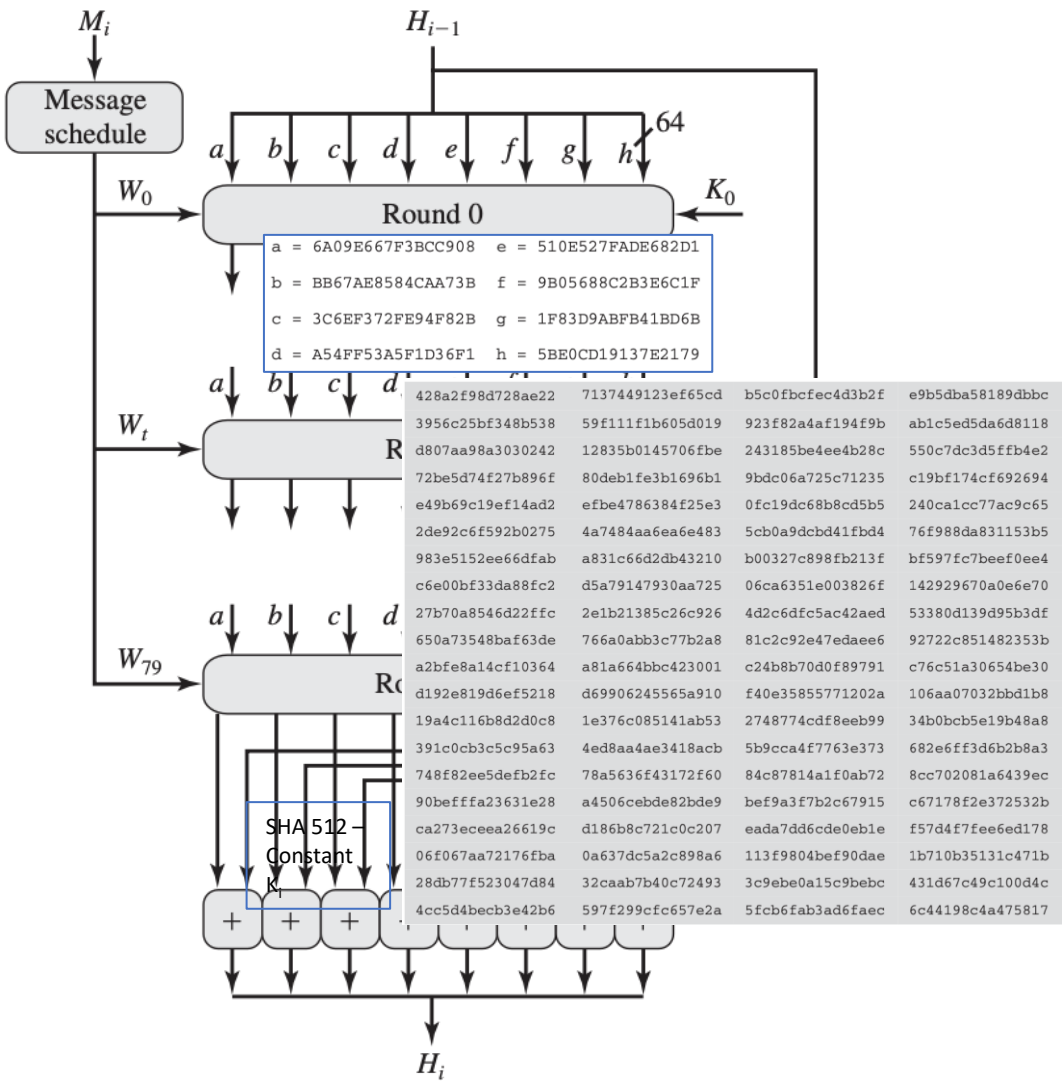


Secure Hash Algorithm (SHA)

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80



Secure Hash Algorithm – SHA 512



428a2f98d728ae22	7137449123ef65cd	b5c0fbcfec4d3b2f	e9b5dba58189dbbc
3956c25bf348b538	59f111f1b605d019	923f82a4af194f9b	ab1c5ed5da6d8118
d807aa98a3030242	12835b0145706fbe	243185be4ee4b28c	550c7dc3d5ffb4e2
72be5d74f27b896f	80deb1fe3b1696b1	9bdc06a725c71235	c19bf174cf692694
e49b69c19ef14ad2	efbe4786384f25e3	0fc19dc68b8cd5b5	240ca1cc77ac9c65
2de92c6f592b0275	4a7484aa6ea6e483	5cb0a9dcbd41fbd4	76f988da831153b5
983e5152ee66dfab	a831c66d2db43210	b00327c898fb213f	bf597fc7beef0ee4
c6e00bf33da88fc2	d5a79147930aa725	06ca6351e003826f	142929670a0e6e70
27b70a8546d22ffc	2e1b21385c26c926	4d2c6dfc5ac42aed	53380d139d95b3df
650a73548baf63de	766a0abb3c77b2a8	81c2c92e47edaee6	92722c851482353b
a2bfe8a14cf10364	a81a664bbc423001	c24b8b70d0f89791	c76c51a30654be30
d192e819d6ef5218	d69906245565a910	f40e35855771202a	106aa07032bbd1b8
19a4c116b8d2d0c8	1e376c085141ab53	2748774cdf8eeb99	34b0bcb5e19b48a8
391c0cb3c5c95a63	4ed8aa4ae3418acb	5b9cca4f7763e373	682e6ff3d6b2b8a3
748f82ee5defb2fc	78a5636f43172f60	84c87814a1f0ab72	8cc702081a6439ec
90befffa23631e28	a4506cebde82bde9	bef9a3f7b2c67915	c67178f2e372532b
ca273eceea26619c	d186b8c721c0c207	eada7dd6cde0eb1e	f57d4f7fee6ed178
06f067aa72176fba	0a637dc5a2c898a6	113f9804bef90dae	1b710b35131c471b
28db77f523047d84	32caab7b40c72493	3c9ebe0a15c9bebc	431d67c49c100d4c
4cc5d4becb3e42b6	597f299cfc657e2a	5fcb6fab3ad6faec	6c44198c4a475817

Authenticated Encryption

Goals

An **authenticated encryption** system (E,D) is a cipher where

As usual: $E: K \times M \times N \longrightarrow C$

but $D: K \times C \times N \longrightarrow M \cup \{\perp\}$

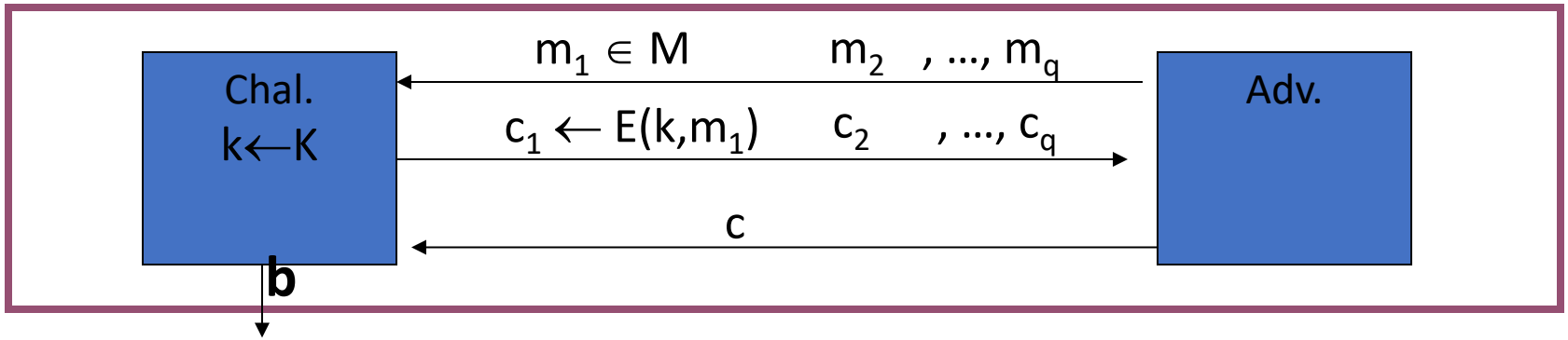
<u>Security</u>: hệ thống phải đảm bảo

ciphertext is rejected

- sem. security under a CPA attack, and
- **ciphertext integrity:**
  kẻ tấn công không thể tạo ra ciphertexts mới không thể giải mã chính xác

Toàn vẹn Ciphertext

Let (E,D) be a cipher with message space M.



$$\begin{cases} \mathbf{b}=1 & \text{if } D(k,c) \neq \perp \text{ and } c \notin \{c_1, \ldots, c_q\} \\ \mathbf{b}=0 & \text{otherwise} \end{cases}$$

Def: (E,D) has **<u>ciphertext integrity</u>** if for all "efficient" A:

$$\mathrm{Adv}_{\mathrm{CI}}[A,E] = \Pr[\text{Chal. outputs } 1]$$ is "negligible."

Mã xác thực (Authenticated encryption)

Def: Hệ mã (E,D) tạo ra **<u>authenticated encryption</u> (AE)** nếu là

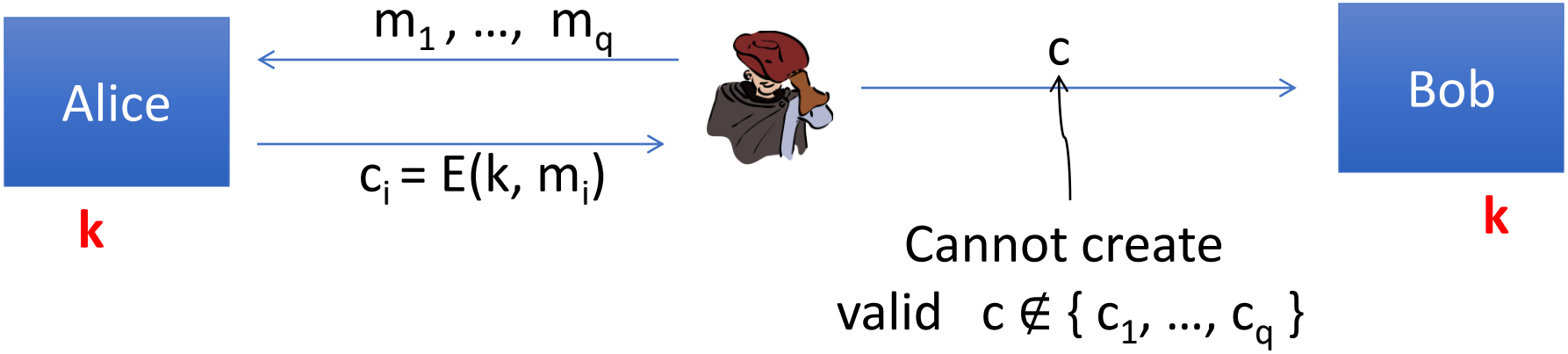
(1) bảo mật ngữ nghĩa với tấn công CPA

(2) có bản mã toàn vẹn

Bad example: CBC with rand. IV does not provide AE

- D(k,·) không tạo ra đầu ra ⊥, do đó adv. dễ dàng chiến thắng CI game

Kẻ tấn công không thể đánh lừa Bob nghĩ rằng một tin nhắn được gửi từ Alice



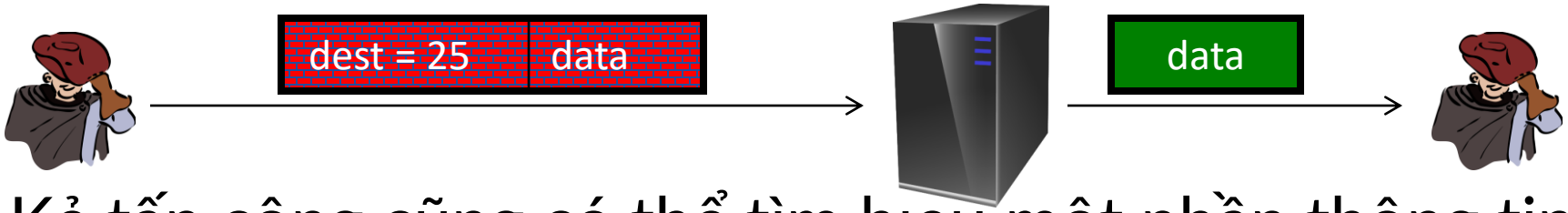
⇒ nếu $D(k,c) \neq \perp$ Bob biết tin nhắn là từ một người biết về K (but message could be a replay)

Authenticated encryption ⇒ Security against **chosen ciphertext attacks** (next segment)

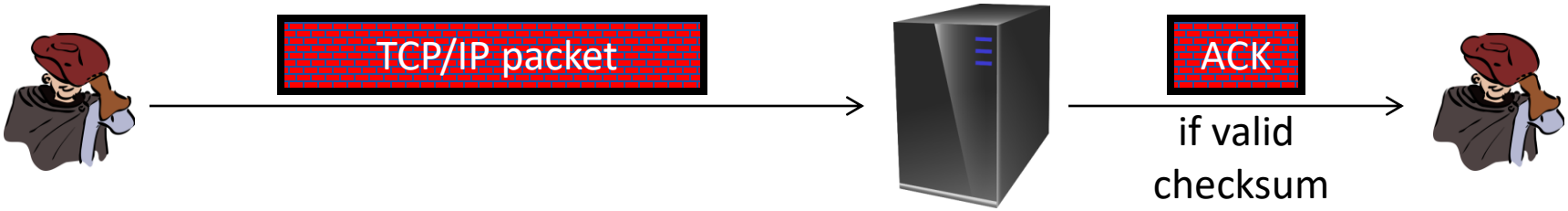
Ví dụ về các tấn công CCP (chosen ciphertext

Adversary có ciphertext c muốn giải mã

Thông thường, adv. có thể lừa máy chủ để giải trong việc giải mã một số bản mã nhất định (not c)



- Kẻ tấn công cũng có thể tìm hiểu một phần thông tin của bản rõ



Chosen ciphertext security

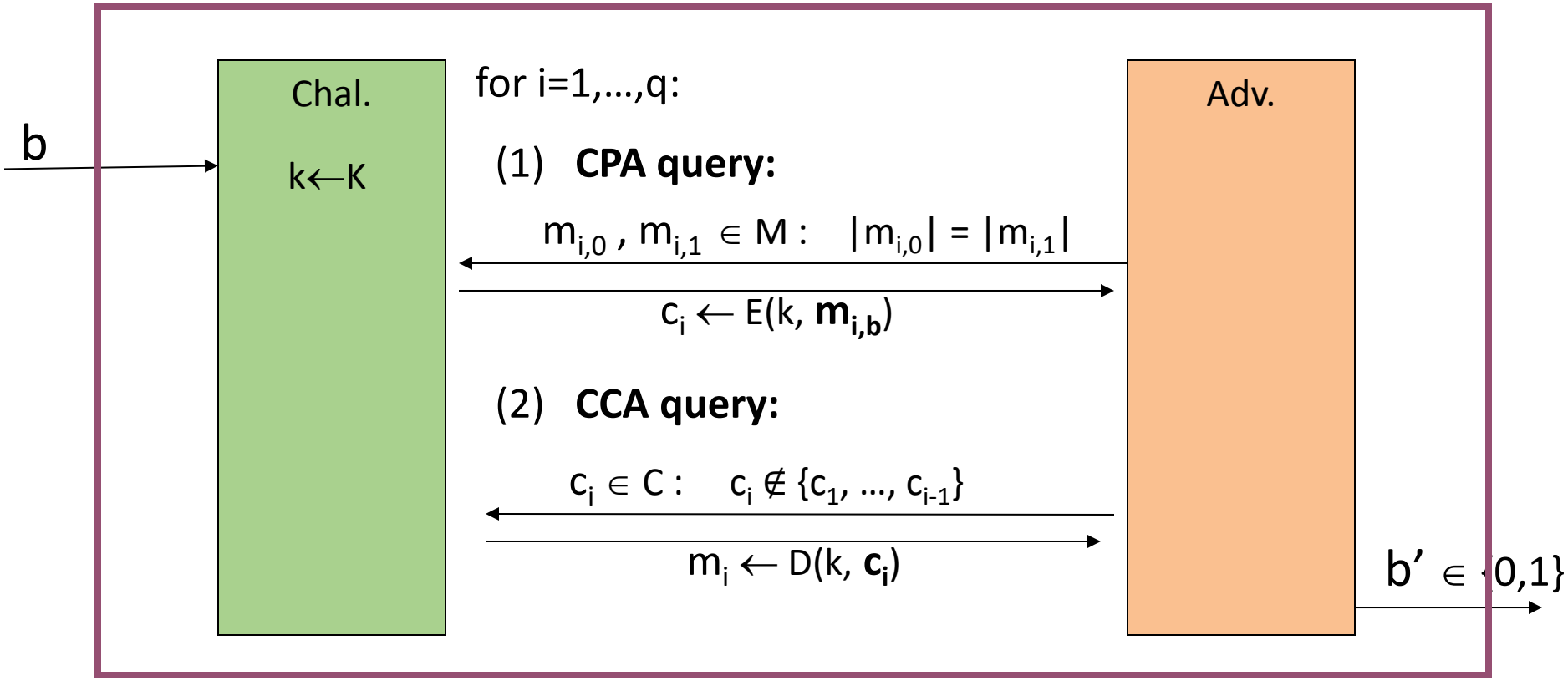
Adversary's power: both CPA and CCA

- Có thể thu được bản mã của các thông điệp tuỳ ý theo sự lựa chọn của kẻ tấn công
- Có thể giải mã bất kỳ bản mã nào do kẻ tấn công lựa chọn

Adversary's goal: Break sematic security

Chosen ciphertext security: definition

$\mathbb{E}$ = (E,D) cipher defined over (K,M,C). For b=0,1 define EXP(b):

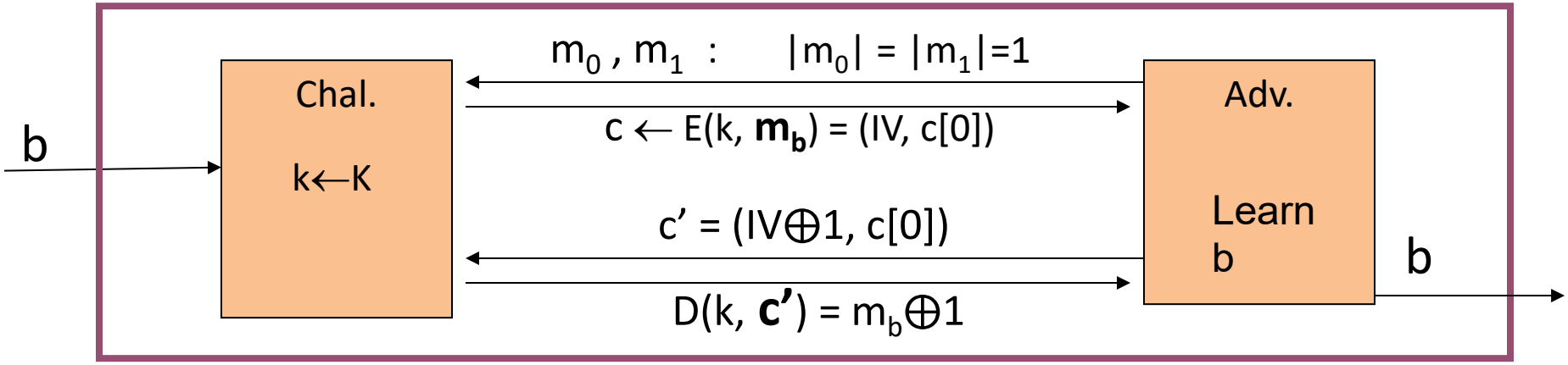


Chosen ciphertext security: definition

$\mathbb{E}$ is CCA secure if for all "efficient" A:

$\mathrm{Adv}_{CCA}[A,\mathbb{E}] = |\Pr[EXP(0)=1] - \Pr[EXP(1)=1]|$ is "negligible."

Example: CBC with rand. IV is not CCA-secure



… but first, some history

Authenticated Encryption (AE): introduced in 2000
[KY'00, BN'00]

Crypto APIs before then: (e.g. MS-CAPI)

- Provide API for CPA-secure encryption (e.g. CBC with rand. IV)
- Provide API for MAC (e.g. HMAC)

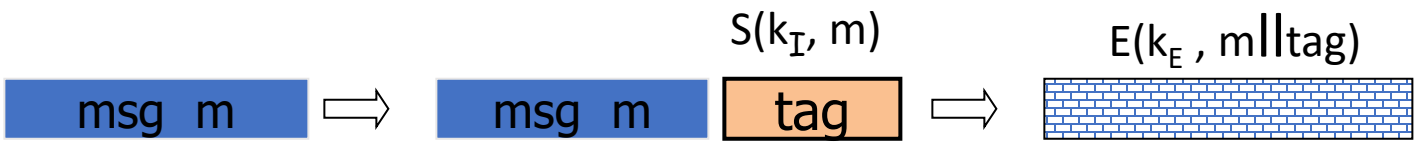
Every project had to combine the two itself without a well defined goal

- Not all combinations provide AE …

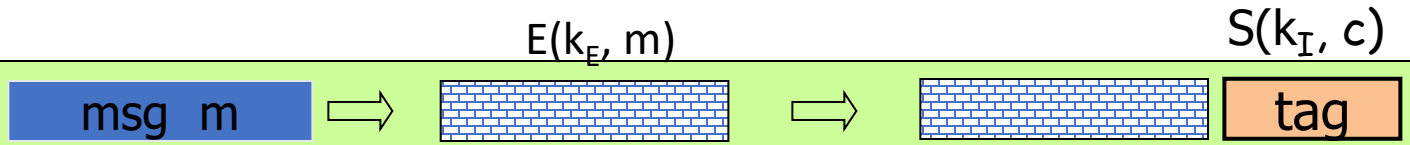
Combining MAC and ENC (CCA)

Encryption key k_E. MAC key = k_I

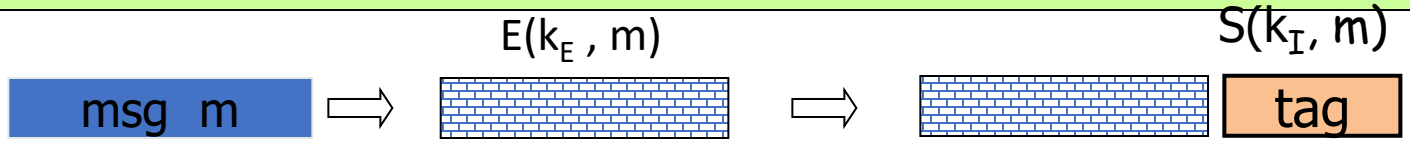
Option 1: (SSL)



Option 2: (IPsec)



Option 3: (SSH)



always correct

A.E. Theorems

Let (E,D) be CPA secure cipher and (S,V) secure MAC. Then:

1. **Encrypt-then-MAC**: always provides A.E.

2. **MAC-then-encrypt**: may be insecure against CCA attacks

   however: when (E,D) is rand-CTR mode or rand-CBC

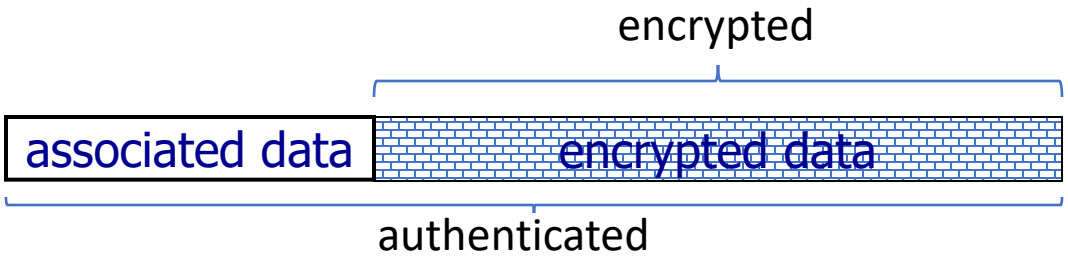
   M-then-E provides A.E.

   for rand-CTR mode, one-time MAC is sufficient

Standards (at a high level)

- **GCM**: CTR mode encryption then CW-MAC
  (accelerated via Intel's PCLMULQDQ instruction)
- **CCM**: CBC-MAC then CTR mode encryption
  (802.11i)
- **EAX**: CTR mode encryption then CMAC

All support AEAD: (auth. enc. with associated data). All are nonce-based.

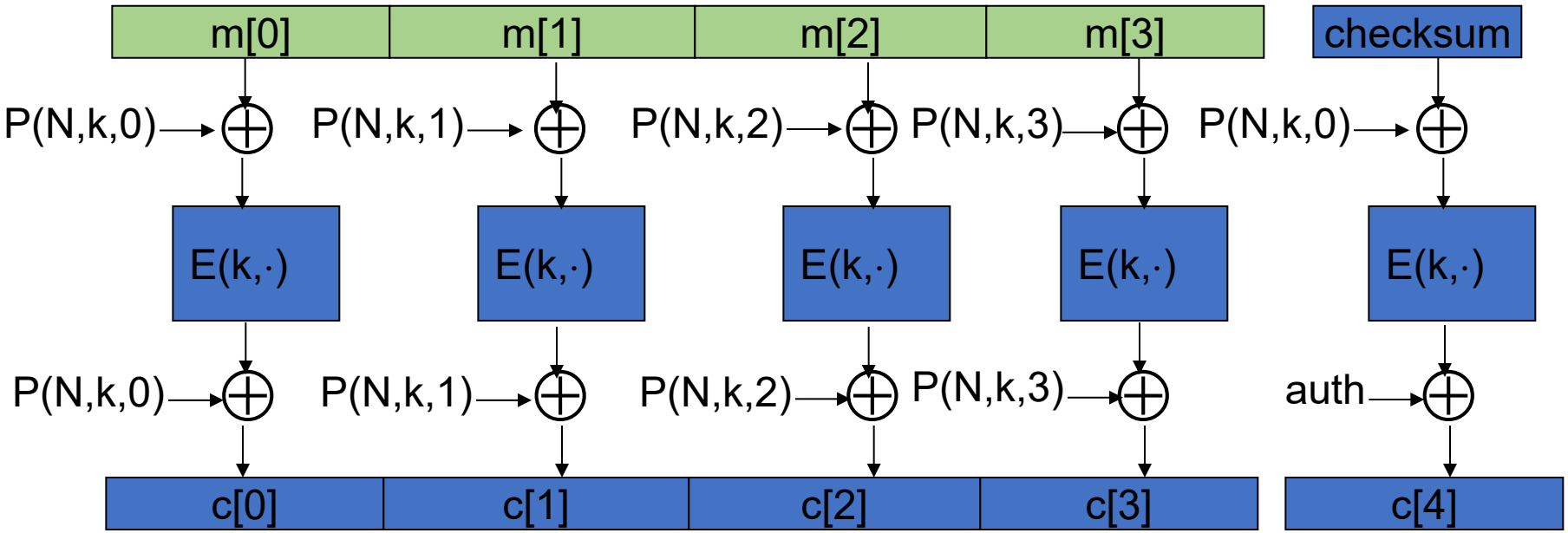


An example API (OpenSSL)

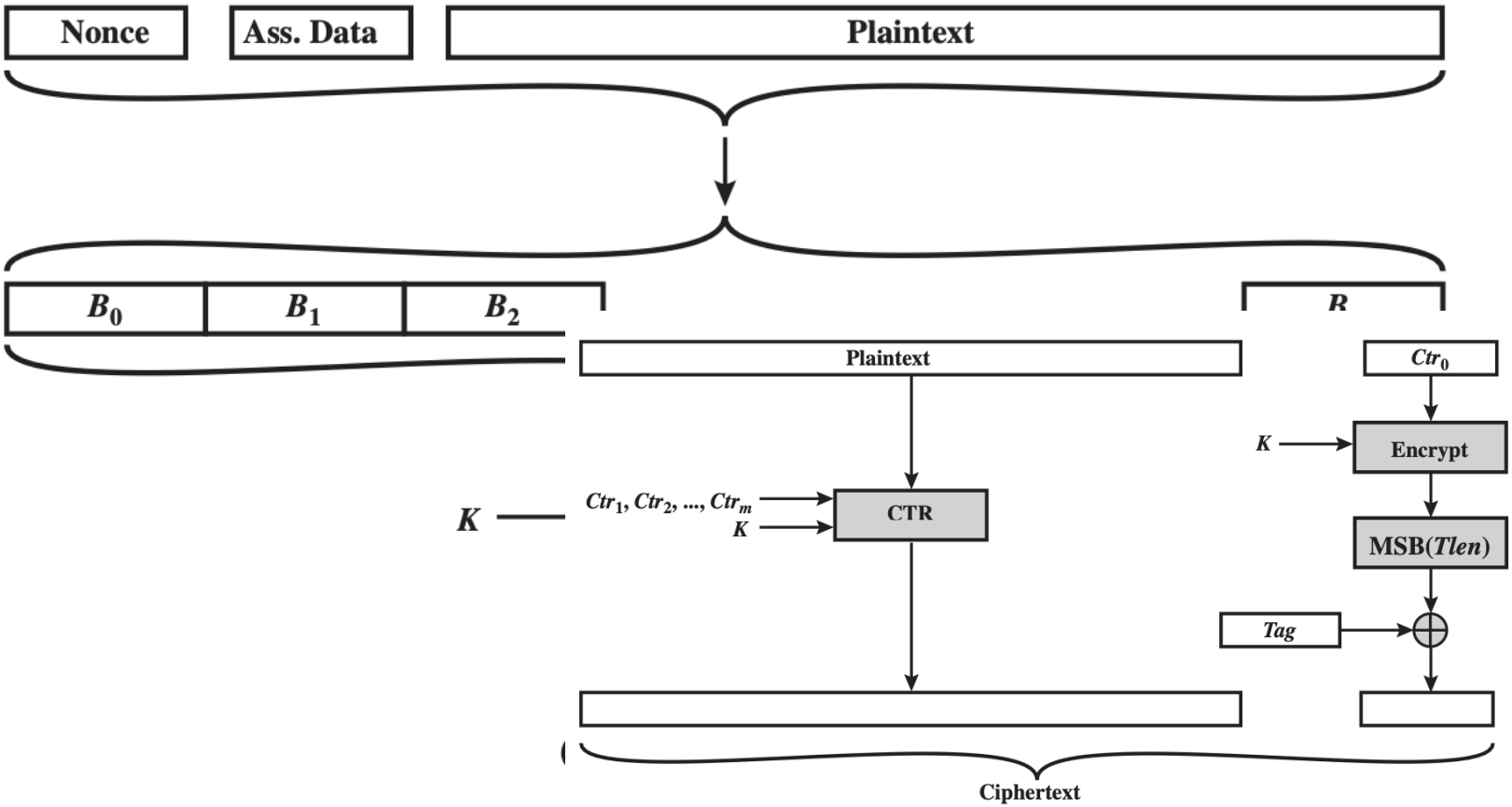
```
int AES_GCM_Init(AES_GCM_CTX *ain,
        unsigned char *nonce,   unsigned long noncelen,
        unsigned char *key,   unsigned int klen )

int AES_GCM_EncryptUpdate(AES_GCM_CTX *a,
        unsigned char *aad,   unsigned long aadlen,
        unsigned char *data,   unsigned long datalen,
        unsigned char *out,   unsigned long *outlen)
```

OCB: a direct construction from a PRP

More efficient authenticated encryption: one E() op. per block.



CCM – Authenticated Encryption



(b) Encryption

CCM – Authenticated Encryption

SP 800-38C defines the authentication/encryption process as follows.

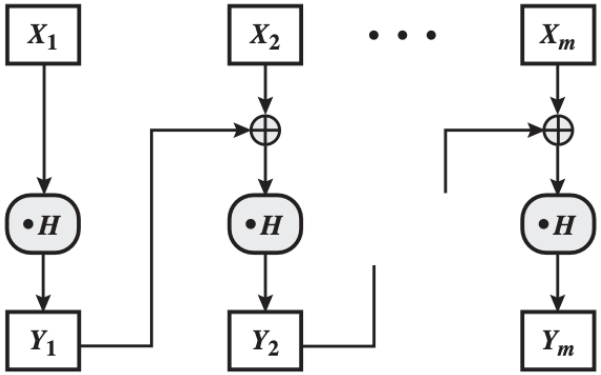
1. Apply the formatting function to (N, A, P) to produce the blocks $B_0, B_1, \ldots, B_r$.
2. Set $Y_0 = E(K, B_0)$.
3. For $i = 1$ to r, do $Y_i = E(K, (B_i \oplus Y_{i-1}))$.
4. Set $T = MSB_{Tlen}(Y_r)$.
5. Apply the counter generation function to generate the counter blocks $Ctr_0, Ctr_1, \ldots, Ctr_m$,where $m = \lceil Plen/128 \rceil$
6. For $j = 0$ to m do $S_j = E(K, Ctr_j)$.
7. Set $S = S_1 || S_2 || \ldots || S_m$.
8. Return $C = (P \oplus MSB_{Plen}(S)) \,||\, (T \oplus MSB_{Tlen}(S_0))$.

CCM – Authenticated Encryption

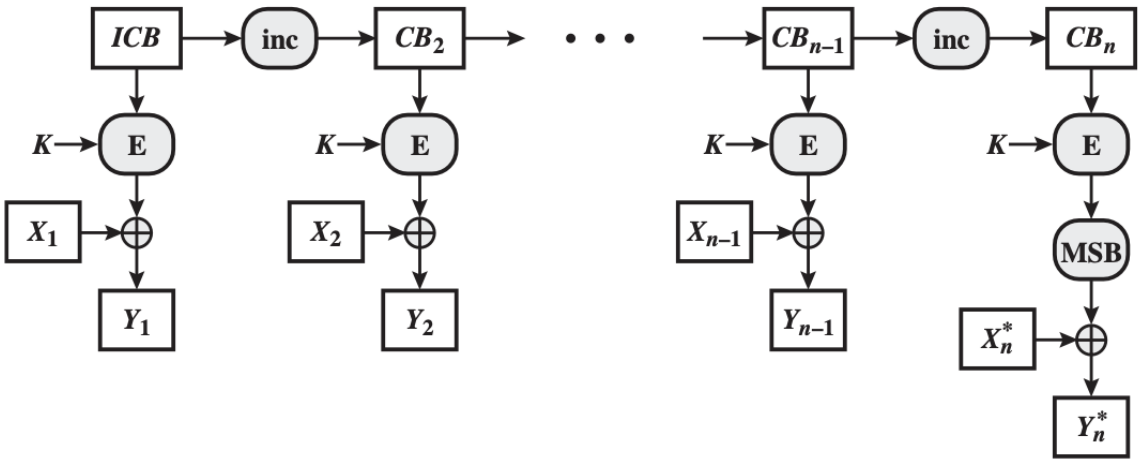
For decryption and verification, the recipient requires the following input: the ciphertext C, the nonce N, the associated data A, the key K, and the initial counter Ctr_0. The steps are as follows.

1. If $Clen \leq Tlen$, then return INVALID.
2. Apply the counter generation function to generate the counter blocks Ctr_0 , Ctr_1, … , Ctr_m, where $m = [Clen/128]$
3. For $j = 0$ to m, do $S_j = E(K,Ctr_j)$.
4. Set $S = S_1||S_2||...||S_m$.
5. Set $P = MSB_{Clen-Tlen}(C) \oplus MSB_{Clen-Tlen}(S)$.
6. Set $T = LSB_{Tlen}(C) \oplus MSB_{Tlen}(S_0)$.
7. Apply the formatting function to (N, A, P) to produce the blocks $B_0, B_1, ..., B_r$.
8. Set $Y_0 = E(K, B_0)$.
9. For $i = 1$ to r, do $Y_i = E(K, (B_i \oplus Y_{i-1}))$.
10. If $T \neq MSB_{Tlen}(Y_r)$, then return INVALID, else return P.

GCM – Authenticated Encryption

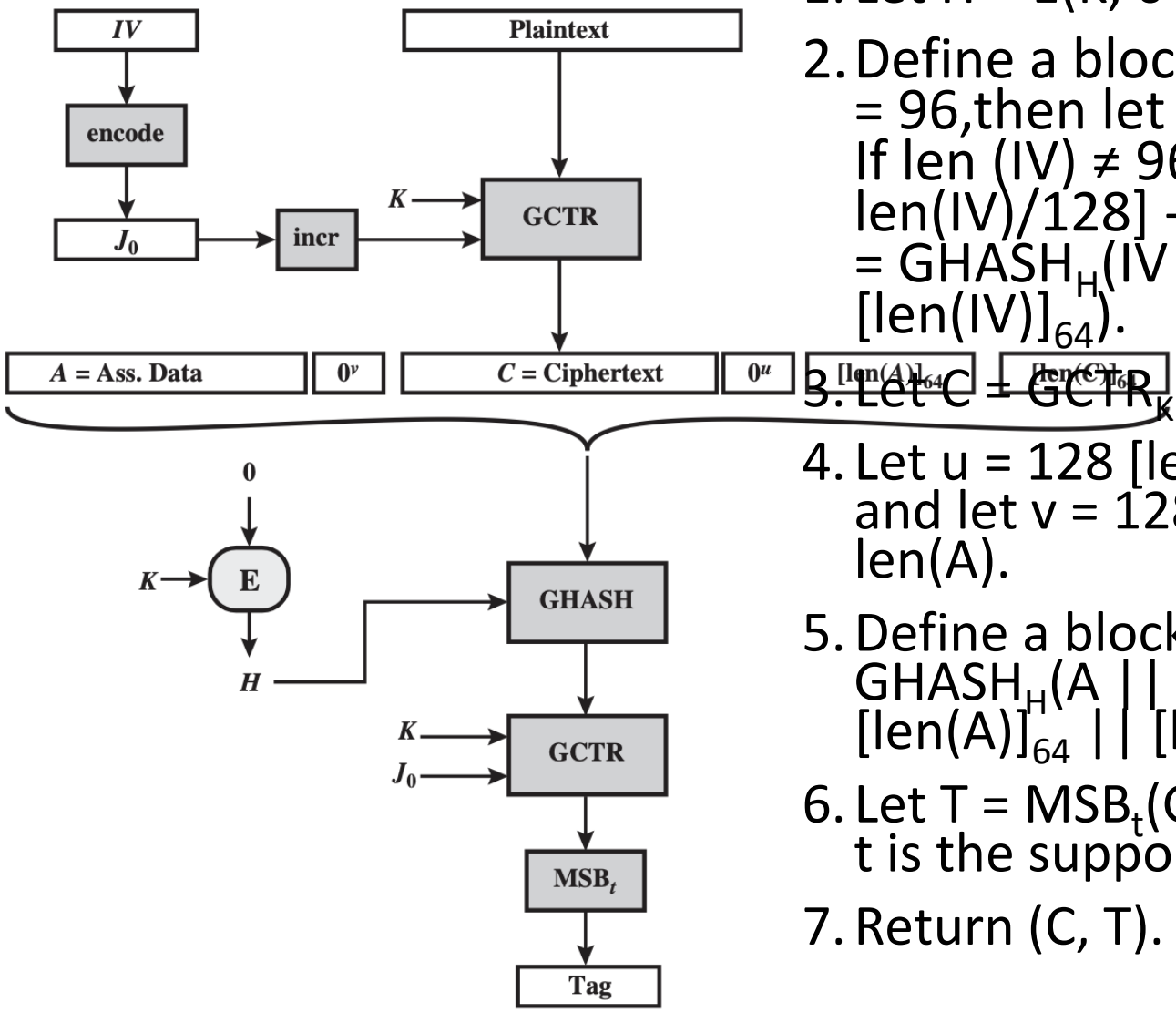


(a) $\text{GHASH}_H(X_1 \| X_2 \| \ldots \| X_m) = Y_m$



(b) $\text{GCTR}_K(ICB, X_1 \| X_2 \| \ldots \| X_n^*) = Y_n^*$

Galois Counter— Message Authentication Code (GCM)



1. Let $H = E(K, 0^{128})$.
2. Define a block, J_0, as : If len(IV) = 96,then let $J_0 = IV \,||\, 0^{31} \,||\, 1$. If len (IV) ≠ 96, then let s = 128 [len(IV)/128] - len(IV), and let $J_0 = \text{GHASH}_H(IV \,||\, 0^{s+64} \,||\, [\text{len}(IV)]_{64})$.
3. Let $C = \text{GCTR}_K(\text{inc}_{32}(J_0), P)$.
4. Let u = 128 [len(C)/128] len(C) and let v = 128 [len(A)/128] - len(A).
5. Define a block, S, as $S = \text{GHASH}_H(A \,||\, 0^v \,||\, C \,||\, 0^u \,||\, [\text{len}(A)]_{64} \,||\, [\text{len}(C)]_{64})$
6. Let $T = \text{MSB}_t(\text{GCTR}_K(J_0, S))$, where t is the supported tag length.
7. Return (C, T).

Performance: Crypto++ 5.6.0 [Wei Dai]

AMD Opteron, 2.2 GHz (Linux)

Cipher	code size	Speed (MB/sec)		
AES/GCM	large **	108	AES/CTR	139
AES/CCM	smaller	61	AES/CBC	109
AES/EAX	smaller	61		
			AES/CMAC	109
AES/OCB		129*	HMAC/SHA1	147

* extrapolated from Ted Kravitz's results ** non-Intel machines